UNIVERSIDAD AUTÓNOMA DE GUADALAJARA

Analysis and Design of Systems with Big Data

LABORATORY REPORT #4

Storing and Analyzing Data by Using Amazon Redshift

Student Name: Jorge Labrada Ramos

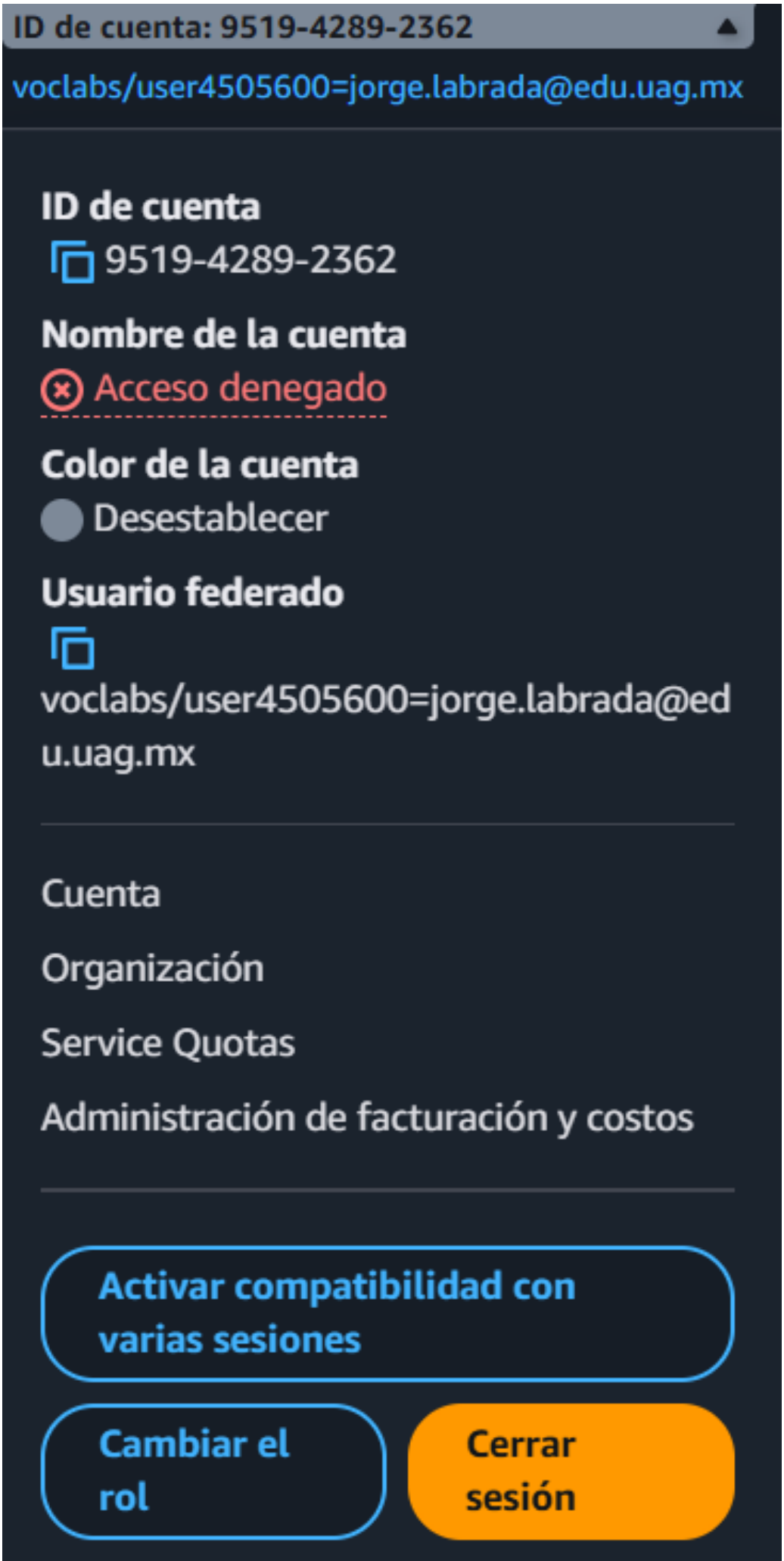
Student ID: 4994545

Professor: Agustín Villarreal

Submission Date: 03/07/2026

1 AWS Account Information

AWS Account ID: 9519-4289-2362


ID de cuenta: 9519-4289-2362
voclabs/user4505600=jorge.labrada@edu.uag.mx
ID de cuenta
9519-4289-2362
Nombre de la cuenta
Acceso denegado
Color de la cuenta
Desestablecer
Usuario federado
voclabs/user4505600=jorge.labrada@edu.uag.mx
Cuenta
Organización
Service Quotas
Administración de facturación y costos
Activar compatibilidad con varias sesiones
Cambiar el rol
Cerrar sesión

AWS Region Used: us-east-1 (US East — N. Virginia)

2 Laboratory Objectives

General Objective

Implement an Amazon Redshift cluster as part of an analytics pipeline, loading data from Amazon S3 and running SQL queries to analyze large datasets.

Specific Objectives

- Review and understand the IAM role permissions required to access and configure Amazon Redshift.
- Create and configure a Redshift cluster with its corresponding security groups.
- Load data from Amazon S3 into Redshift tables and execute analytical SQL queries.

Competencies Developed

- Configuration and administration of columnar data warehouses in the cloud using Amazon Redshift.
- Use of the AWS CLI and the Redshift Data API to programmatically interact with databases.
- Management of IAM permissions and policies to control access to AWS services.

3 Laboratory Development

3.1 Task 1: Reviewing the IAM Role for Amazon Redshift

The IAM console was accessed to review the `MyRedshiftRole` role, which has two policies attached: `AmazonS3ReadOnlyAccess` and `RedshiftIAMLabPolicy`. The role ARN was copied to be used later in the COPY commands.

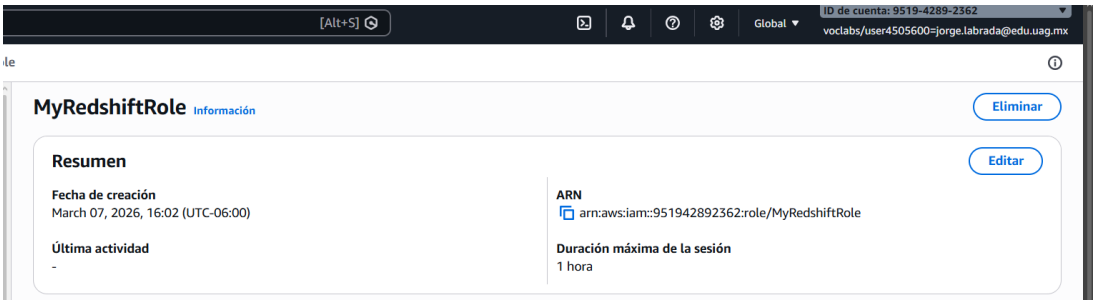


Figure 2: Summary of the MyRedshiftRole IAM role with ARN: arn:aws:iam::951942892362:role/MyRedshiftRole

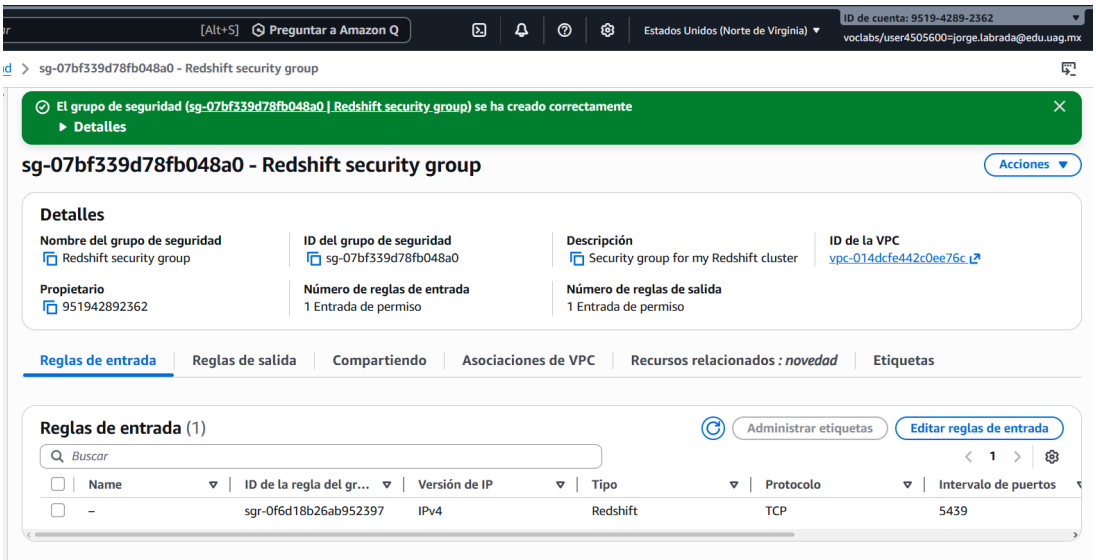


Figure 3: Confirmation of ARN copy and the two permission policies attached to MyRedshiftRole

The role ARN is:

```
arn:aws:iam::951942892362:role/MyRedshiftRole
```

3.2 Task 2: Creating and Configuring the Redshift Cluster

The cluster `redshift-cluster-1` was created with the following configuration:

Parameter	Value	Justification
Cluster identifier	redshift-cluster-1	Required name for the lab
Node type	ra3.large	Cost-performance balance for the POC
Number of nodes	2	Leader node + compute node
Admin user	awsuser	Default lab admin user

Table 1: Redshift cluster configuration parameters

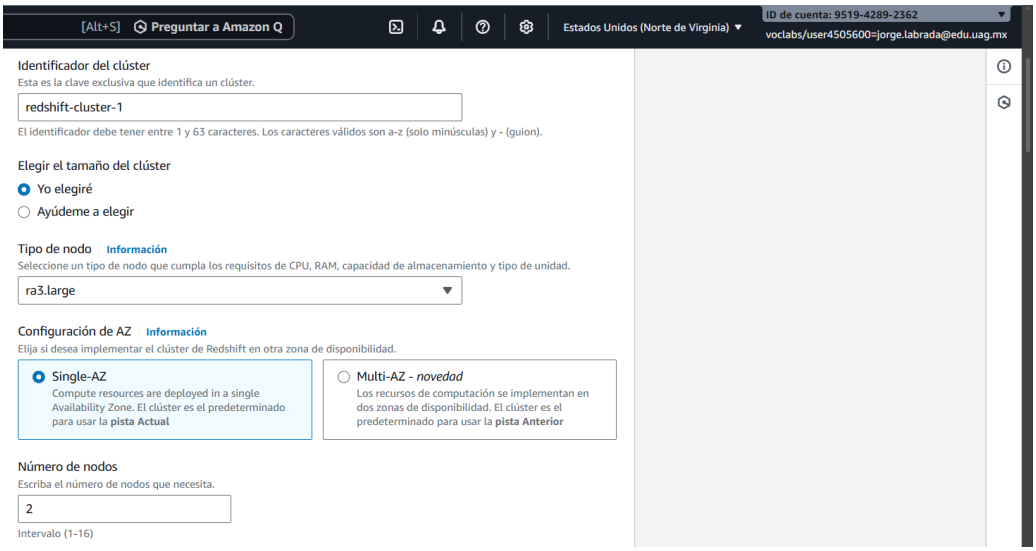


Figure 4: Cluster identifier set to redshift-cluster-1, node type ra3.large, and 2 nodes

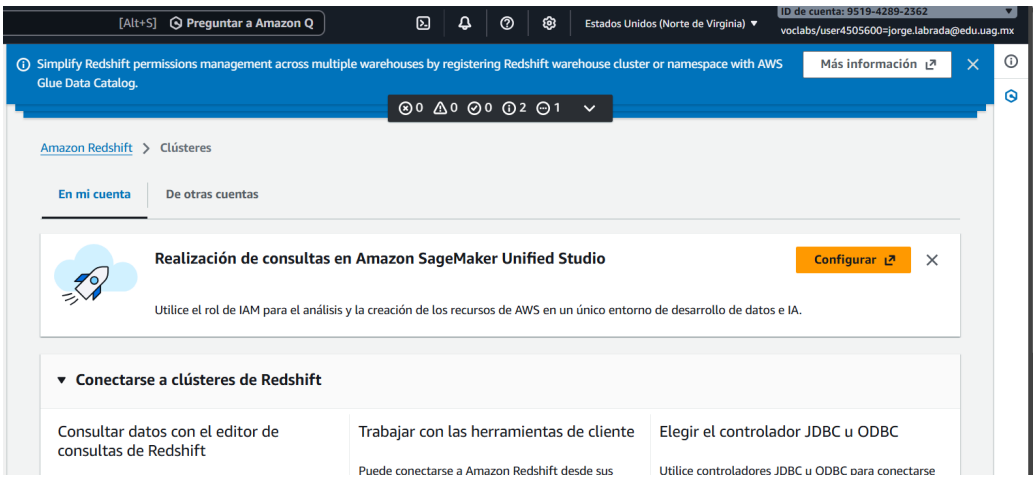


Figure 5: Amazon Redshift console — clusters view during the creation process

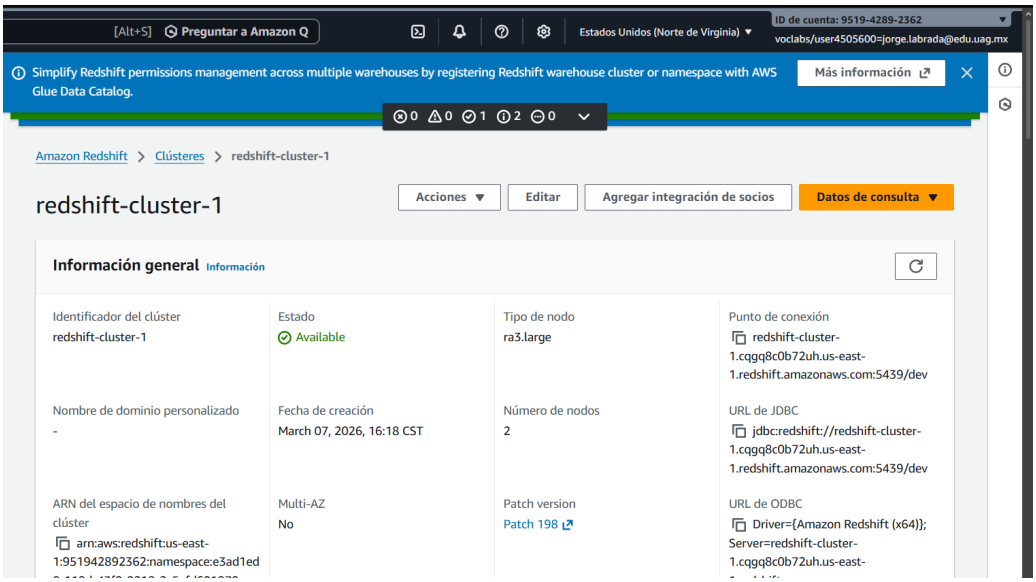


Figure 6: MyRedshiftRole IAM role associated to the cluster with in-sync status

A security group named **Redshift security group** was also created in EC2 with an inbound rule on TCP port 5439 (the default Redshift port), allowing access from `0.0.0.0/0`.

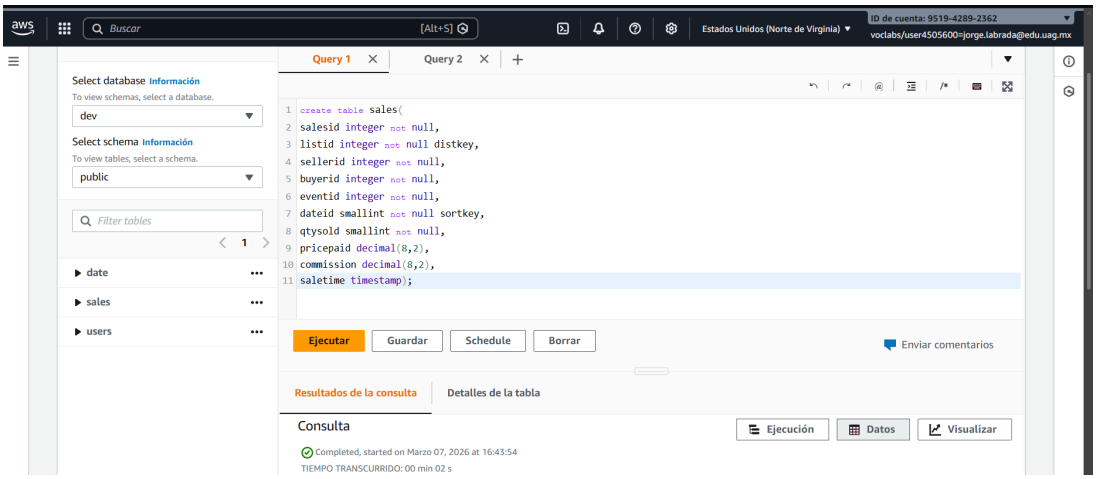


Figure 7: Security group successfully created with inbound rule on port 5439 (Redshift/TCP)

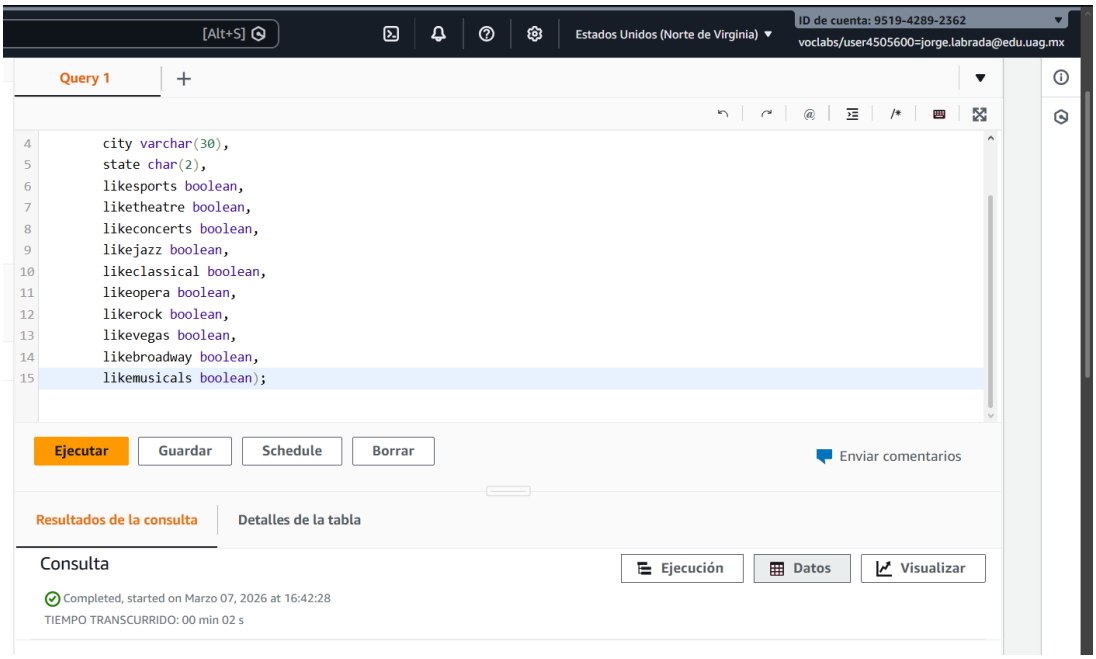


Figure 8: Cluster redshift-cluster-1 with status Available — general information panel

3.3 Task 3: Creating Tables in the Database

The Redshift Query Editor was used to create the `users`, `date`, and `sales` tables in the `dev` database under the `public` schema.

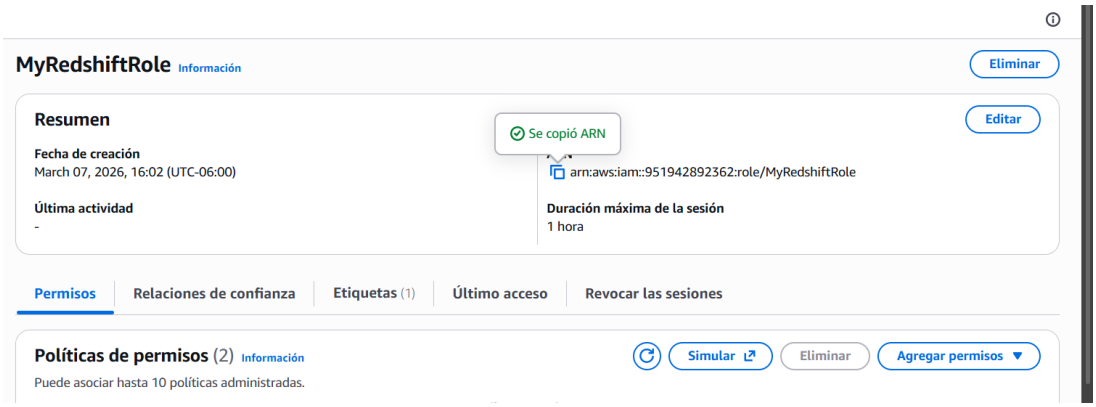


Figure 9: Creation of the users table — query completed successfully (2 seconds)

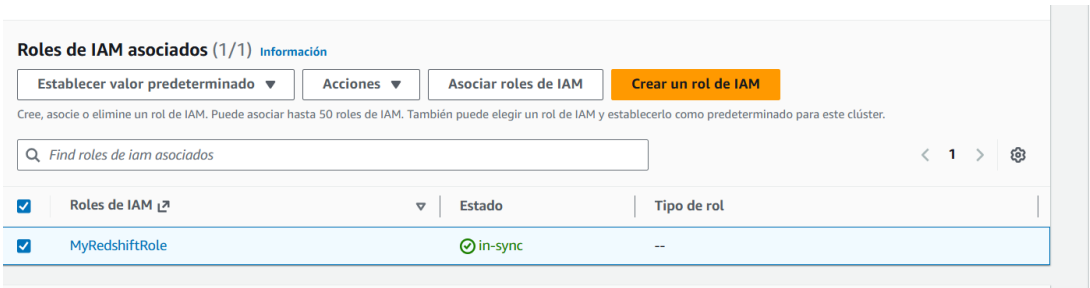


Figure 10: Creation of the sales table with all columns and data types visible in the editor

The SQL used to create the three tables:

Listing 1: CREATE TABLE – users

```sql
create table users(
    userid integer not null distkey sortkey,
    username char(8),
    city varchar(30),
    state char(2),
    likesports boolean,
    liketheatre boolean,
    likeconcerts boolean,
    likejazz boolean,
    likeclassical boolean,
    likeopera boolean,
    likerock boolean,
    likevegas boolean,
    likebroadway boolean,
    likemusicals boolean);
```

Listing 2: CREATE TABLE – date

```sql
create table date(
    dateid smallint not null distkey sortkey,
    caldate date not null,
    day character(3) not null,
    week smallint not null,
    month character(5) not null,
    qtr character(5) not null,
    year smallint not null,
    holiday boolean default('N'));
```

Listing 3: CREATE TABLE – sales

```sql
create table sales(
    salesid integer not null,
    listid integer not null distkey,
    sellerid integer not null,
    buyerid integer not null,
    eventid integer not null,
    dateid smallint not null sortkey,
    qtysold smallint not null,
    pricepaid decimal(8,2),
    commission decimal(8,2),
```

```
    saletime timestamp);
```

3.4 Task 4: Loading Data from Amazon S3

The `COPY` command was used three times to load data from a public S3 bucket into each table. The IAM role ARN was provided for authentication.

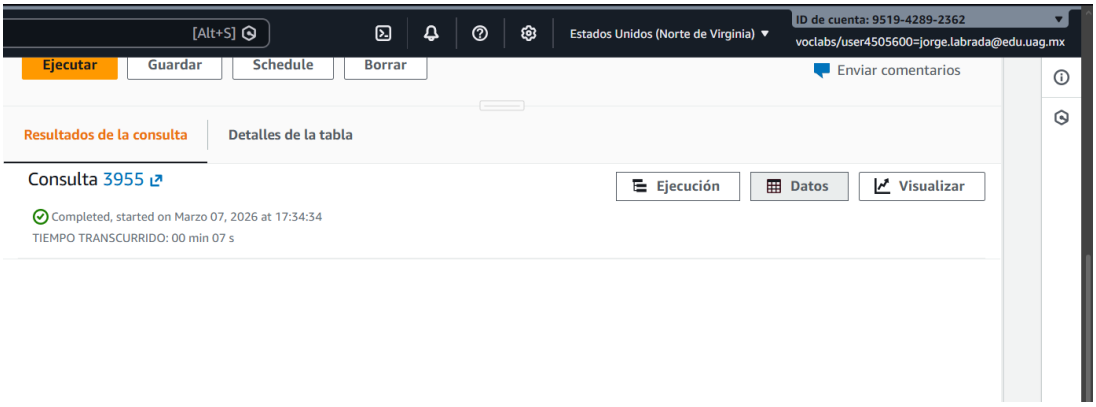


Figure 11: COPY into users table from S3 — Completed (Query 3955, elapsed time: 7 seconds)

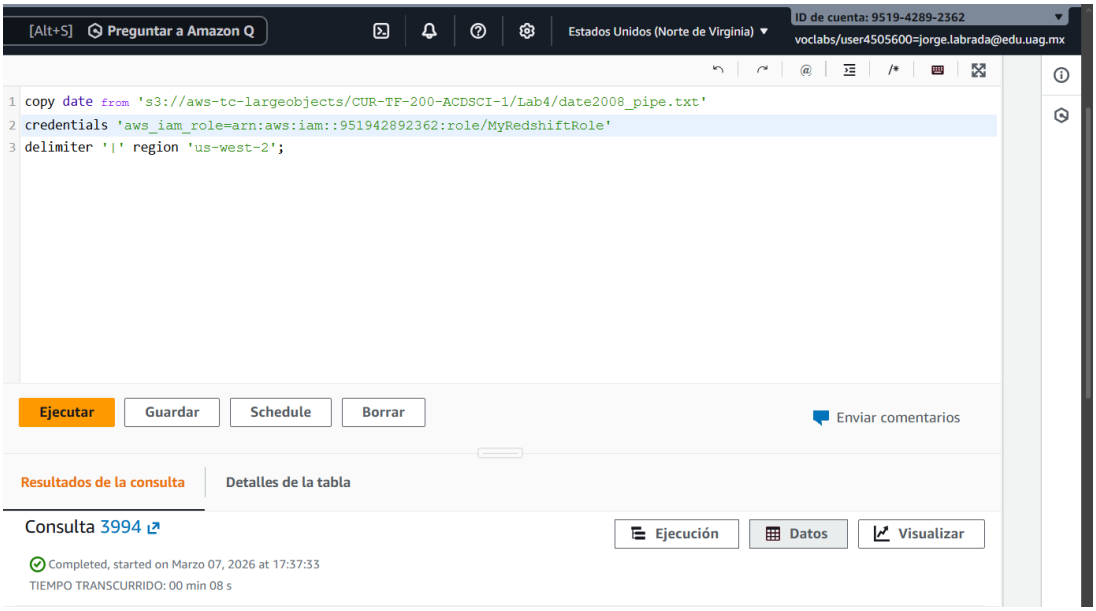


Figure 12: COPY into date table using pipe delimiter — Completed (Query 3994, 8 seconds)

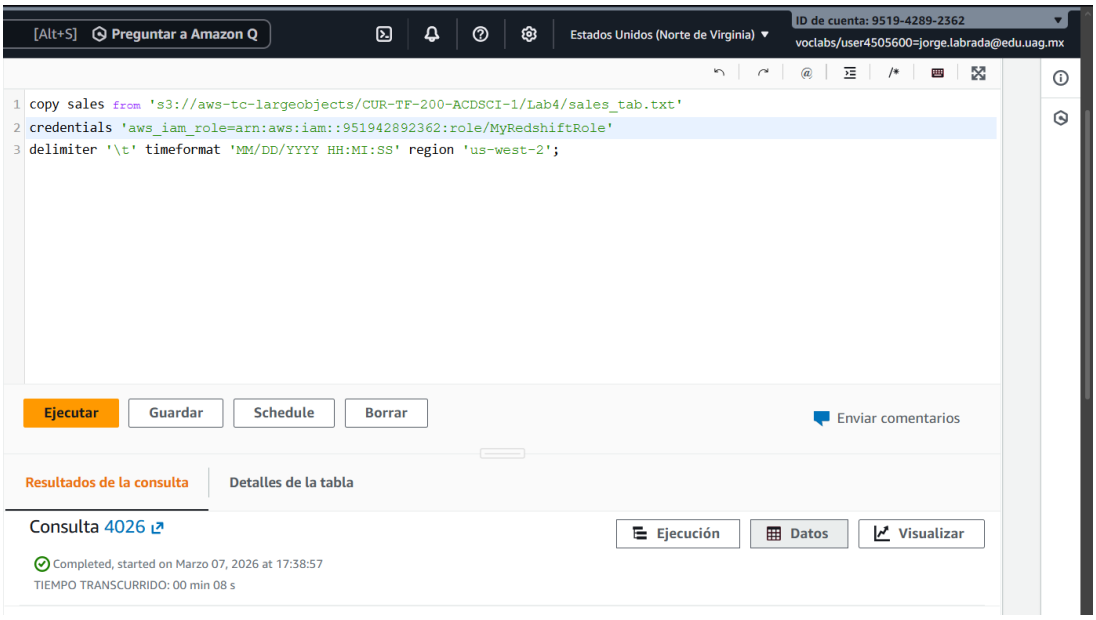


Figure 13: COPY into sales table with timeformat parameter — Completed (Query 4026, 8 seconds)

Listing 4: COPY commands used to load data from S3

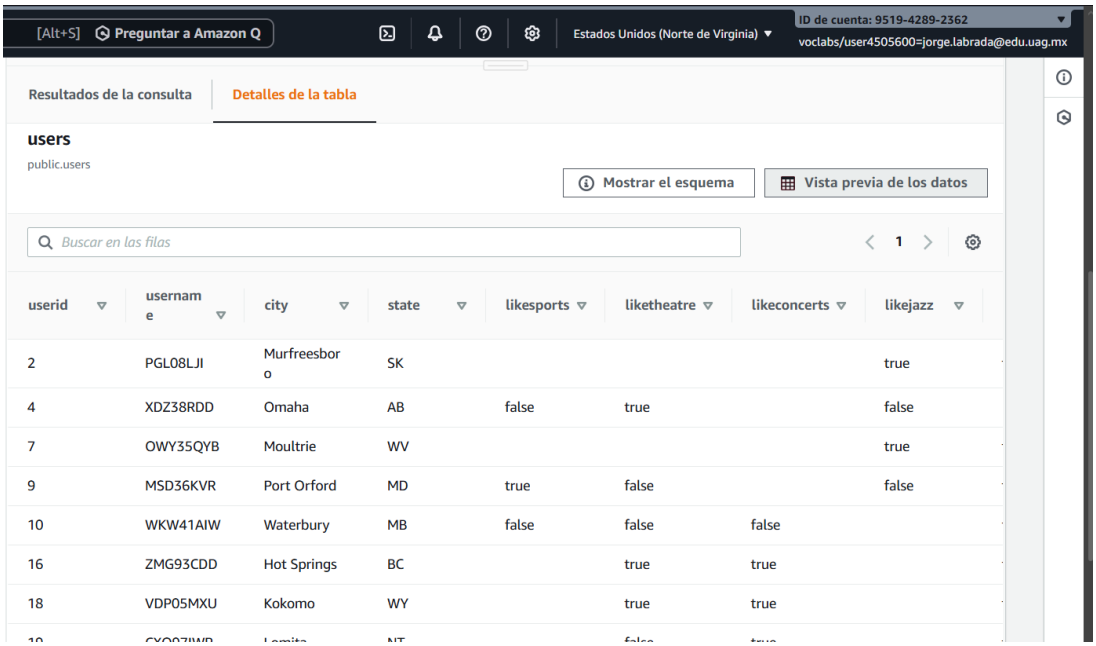
```
-- users table (tab-delimited)
copy users
from 's3://aws-tc-largeobjects/CUR-TF-200-ACDSCI-1/Lab4/allusers_tab.txt'
credentials 'aws_iam_role=arn:aws:iam::951942892362:role/MyRedshiftRole'
delimiter '\t' region 'us-west-2';

-- date table (pipe-delimited)
copy date
from 's3://aws-tc-largeobjects/CUR-TF-200-ACDSCI-1/Lab4/date2008_pipe.txt'
credentials 'aws_iam_role=arn:aws:iam::951942892362:role/MyRedshiftRole'
delimiter '|' region 'us-west-2';

-- sales table (tab-delimited with timestamp format)
copy sales
from 's3://aws-tc-largeobjects/CUR-TF-200-ACDSCI-1/Lab4/sales_tab.txt'
credentials 'aws_iam_role=arn:aws:iam::951942892362:role/MyRedshiftRole'
delimiter '\t' timeformat 'MM/DD/YYYY HH:MI:SS' region 'us-west-2';
```

3.5 Task 5: Querying the Data

Two SQL queries were executed on the `dev` database to validate the loaded data and generate analytical results.

Query 1 — Total items sold on a specific date:

```
SELECT sum(qtysold)
FROM sales, date
WHERE sales.dateid = date.dateid
AND caldate = '2008-01-05';
```



userid	username	city	state	likesports	liketheatre	likeconcerts	likejazz
2	PGL08LJI	Murfreesboro	SK				true
4	XDZ38RDD	Omaha	AB	false	true		false
7	OWY35QYB	Moultrie	WV				true
9	MSD36KVR	Port Orford	MD	true	false		false
10	WKW41AIW	Waterbury	MB	false	false	false	
16	ZMG93CDD	Hot Springs	BC		true	true	
18	VDP05MXU	Kokomo	WY		true	true	

Figure 14: Query result: 210 items sold on 2008-01-05 (1 row returned, elapsed: 1 second)

The query returned the value **210**, representing the total number of items sold on January 5th, 2008.

Query 2 — Top 10 buyers by quantity:

```
SELECT username, total_quantity
FROM
  (SELECT buyerid, sum(qtysold) total_quantity
   FROM sales
   GROUP BY buyerid
   ORDER BY total_quantity desc limit 10) Q, users
WHERE Q.buyerid = userid
ORDER BY Q.total_quantity desc;
```

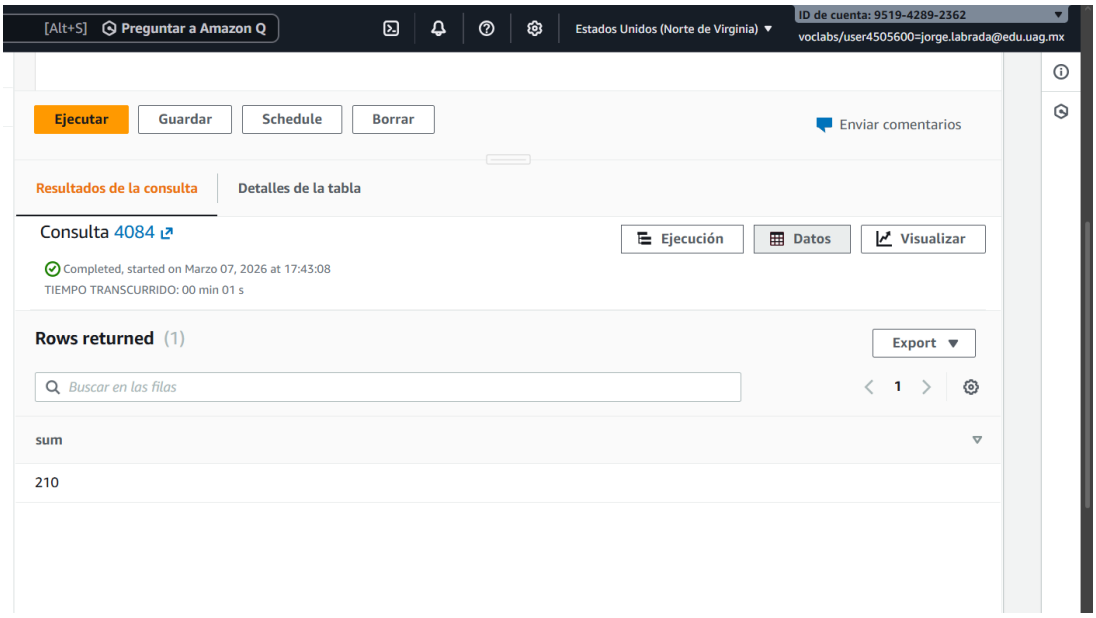


Figure 15: Preview of the users table confirming data was loaded successfully from S3

3.6 Task 6: Running Queries from the AWS CLI

The AWS Cloud9 terminal was used to execute queries against the Redshift cluster via the Redshift Data API.

Listing 5: Executing a query via the AWS CLI

```
aws redshift-data execute-statement \
  --region us-east-1 \
  --db-user awsuser \
  --cluster-identifier redshift-cluster-1 \
  --database dev \
  --sql "select␣*␣from␣users␣limit␣1"
```

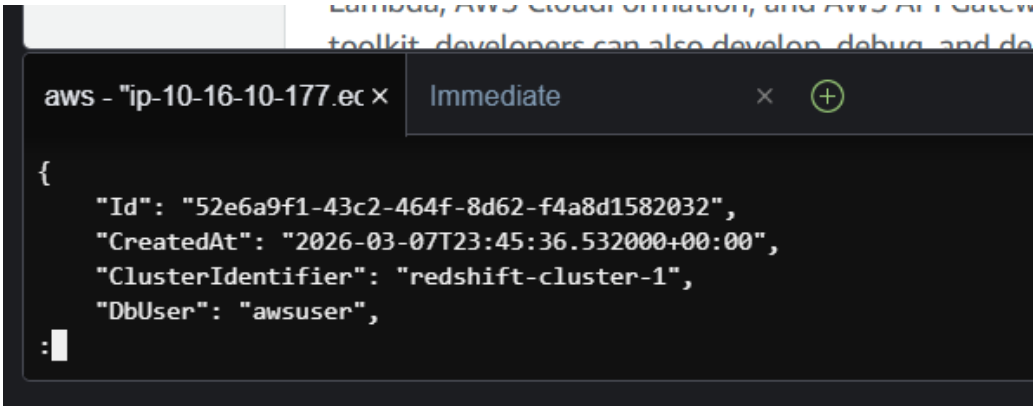


Figure 16: JSON response from execute-statement containing the query ID

Listing 6: Retrieving the query result

```
aws redshift-data get-statement-result \
  --id <QUERY-ID> \
  --region us-east-1
```

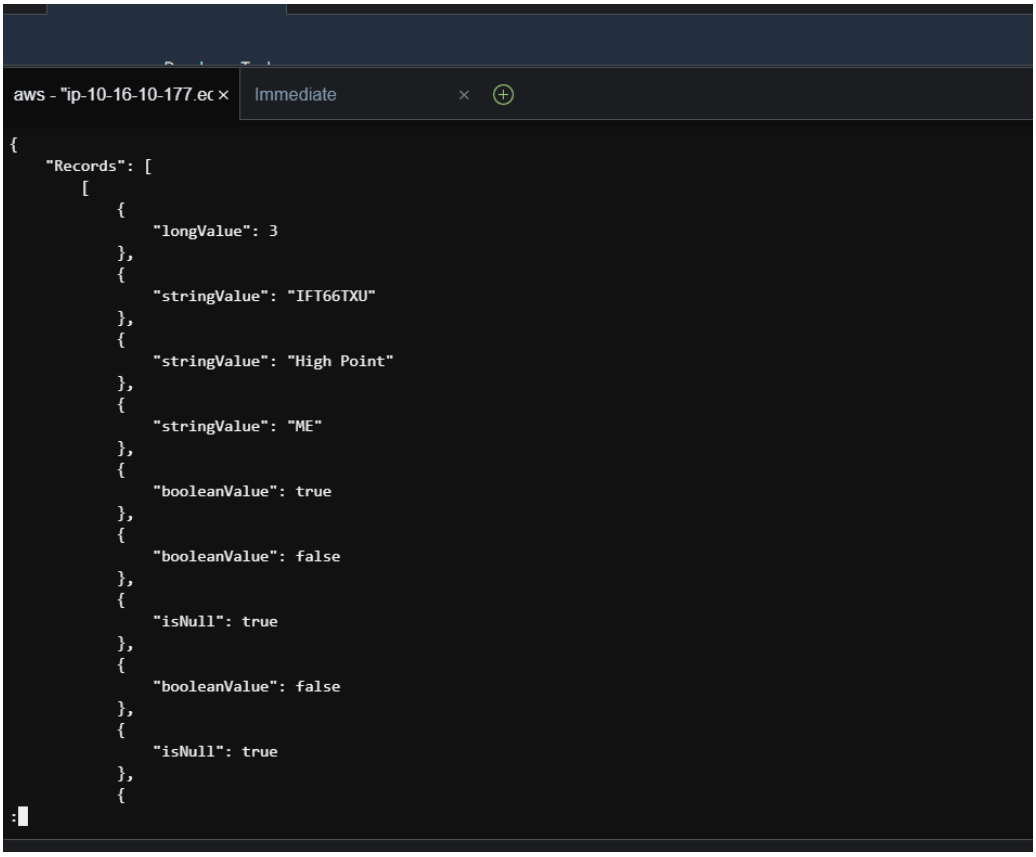


Figure 17: JSON output from get-statement-result showing records from the users table

3.7 Tasks 7 & 8: Reviewing the IAM Policy and Verifying Mary's Access

The `Policy-For-Data-Scientists` IAM policy attached to the `DataScienceGroup` (which Mary belongs to) was reviewed. The policy grants limited access to the Redshift Data API. Mary's credentials were then used to confirm she could successfully execute and retrieve queries.

```
voclabs:~/environment $ AK=AKIA53JBXZNFHPUKUCUK
voclabs:~/environment $ SAK=Ni9HNK366M+JPygA70rDuvJyvpcIzijpWvbeKm+H
voclabs:~/environment $ AWS_ACCESS_KEY_ID=$AK AWS_SECRET_ACCESS_KEY=$SAK aws redshift-d
ter-1 --database dev --sql "select * from users limit 1"
{
    "Id": "a8733a19-e052-40c1-83ef-965857143d70",
    "CreatedAt": "2026-03-07T23:50:57.851000+00:00",
    "ClusterIdentifier": "redshift-cluster-1",
    "DbUser": "awsuser",
    "Database": "dev"
}
voclabs:~/environment $
```

Figure 18: execute-statement executed with Mary's credentials — successful JSON response returned

```
aws - "ip-10-16-10-177.ec ×   Immediate ×
{
    "Records": [
        [
            {
                "longValue": 3
            },
            {
                "stringValue": "IFT66TXU"
            },
            {
                "stringValue": "High Point"
            },
            {
                "stringValue": "ME"
            },
            {
                "booleanValue": true
            },
            {
                "booleanValue": false
            },
            {
                "isNull": true
            },
            {
                "booleanValue": false
            },
            {
                "isNull": true
            },
            {
:
```

Figure 19: get-statement-result output using Mary's credentials — records returned correctly

Listing 7: Verifying Mary's access to the Redshift Data API

```
AK=AKIA53JBXZNFHPUKUCUK
SAK=Ni9HNK366M+JPygA70rDuvJyvpcIzijpWvbeKm+H

AWS_ACCESS_KEY_ID=$AK AWS_SECRET_ACCESS_KEY=$SAK \
aws redshift-data execute-statement \
  --region us-east-1 \
  --db-user awsuser \
  --cluster-identifier redshift-cluster-1 \
  --database dev \
  --sql "select␣*␣from␣users␣limit␣1"
```

4 Results and Validation

4.1 Operational Evidence

Metric	Value
COPY execution time (users)	7 seconds
COPY execution time (date)	8 seconds
COPY execution time (sales)	8 seconds
Query execution time — sum(qtysold)	1 second
Result — sum(qtysold) on 2008-01-05	210
Error rate	0%

Table 2: Performance metrics obtained during the laboratory

5 Analysis and Reflection

5.1 Challenges Encountered

Challenge 1 — Cluster availability wait time:

- **Description:** After creating the cluster, it was necessary to wait several minutes for the status to change to "Available" before continuing with the security group configuration.
- **Solution:** The page was refreshed periodically until the status was confirmed.
- **Learning:** In production environments, automating cluster status verification before executing dependent steps (e.g., using AWS Lambda or Step Functions) would

eliminate this bottleneck.

Challenge 2 — ARN substitution in COPY commands:

- **Description:** The COPY commands required manually replacing the `<REDSHIFT-ROLE-ARN>` placeholder with the actual IAM role ARN before execution.
- **Solution:** The ARN was copied to a text editor and substituted in each command prior to running it.
- **Learning:** Using environment variables or parameterized scripts would eliminate this manual step in automated pipelines.

6 AWS Costs

Service	Usage	Estimated Cost
Amazon Redshift	2x ra3.large nodes, ∼3 hours	$0.57
Amazon S3	Read-only from public bucket	$0.00
AWS Cloud9	t2.micro instance, ∼1.5 hours	$0.02
Amazon EC2 (Security Group)	Network config only	$0.00
TOTAL		**$0.59**

Table 3: AWS cost estimation for this laboratory

Cost optimization strategies applied:

- Used `ra3.large` nodes only for the duration of the lab session.
- Cluster was deleted at the end of the lab to avoid ongoing charges.

7 Conclusions

7.1 Technical Conclusions

1. An Amazon Redshift cluster was successfully deployed with 2 ra3.large nodes. Three tables were populated from Amazon S3 and analytical SQL queries were executed with sub-10-second response times on a music ticket sales dataset.
2. Redshift's columnar storage combined with distribution keys (`distkey`) and sort keys (`sortkey`) significantly optimizes query performance for analytical workloads over large data volumes compared to traditional row-based databases.

3. The IAM, S3, and Redshift integration through assumed IAM roles provides a secure, credential-free architecture — a best practice for AWS workloads.

7.2 Practical Applicability

Amazon Redshift is an ideal solution for data science teams working with multi-gigabyte to petabyte-scale datasets. Its compatibility with standard SQL lowers the adoption barrier, while the Redshift Data API enables automation of analytics pipelines in application code without managing persistent database connections.

7.3 Identified Improvement Areas

- **Area 1:** Implement Redshift Spectrum to query data directly in S3 without loading it into the cluster, reducing storage costs.
- **Area 2:** Automate cluster provisioning and data loading using AWS CloudFormation or CDK for repeatable deployments.
- **Area 3:** Explore Amazon Redshift Serverless for intermittent workloads to eliminate cluster management overhead.

8 References

1. Amazon Web Services, "Amazon Redshift Documentation," AWS Docs. [Online]. Available: https://docs.aws.amazon.com/redshift/ [Accessed: 07-Mar-2026].
2. Amazon Web Services, "Using the Amazon Redshift Data API," AWS Docs. [Online]. Available: https://docs.aws.amazon.com/redshift/latest/mgmt/data-api.html [Accessed: 07-Mar-2026].
3. Amazon Web Services, "Authorizing Amazon Redshift to Access Other AWS Services on Your Behalf," AWS Docs. [Online]. Available: https://docs.aws.amazon.com/redshift/latest/mgmt/authorizing-redshift-service.html [Accessed: 07-Mar-2026].
4. Amazon Web Services, "Amazon Redshift Clusters," AWS Docs. [Online]. Available: https://docs.aws.amazon.com/redshift/latest/mgmt/working-with-clusters.html [Accessed: 07-Mar-2026].
5. Amazon Web Services, "Actions, Resources, and Condition Keys for Amazon Redshift Data API," AWS Docs. [Online]. Available: https://docs.aws.amazon.com/service-authorization/latest/reference/list_amazonredshiftdataapi.html [Accessed: 07-Mar-2026].

9 Code Repository

GitHub Repository:

URL: https://github.com/jorgelabrada/lab4-redshift

Branch: `main`

Commit Hash: `[last relevant commit hash]`

Listing 8: Repository structure

```
lab4-redshift/
|-- README.md
|-- src/
|   |-- create_tables.sql
|   |-- load_data.sql
|   `-- queries.sql
|-- config/
|   `-- cluster_config.json
|-- screenshots/
|   `-- [lab screenshots]
|-- requirements.txt
`-- .gitignore
```

Execution Instructions:

1. Clone the repository: `git clone https://github.com/jorgelabrada/lab4-redshift`
2. Configure AWS credentials: `aws configure`
3. Run SQL scripts in order: `create_tables.sql` → `load_data.sql` → `queries.sql`

A Complete CLI Output — Task 6

Listing 9: execute-statement response (Task 6)

```
{
  "Id": "52e6a9f1-43c2-464f-8d62-f4a8d1582032",
  "CreatedAt": "2026-03-07T23:45:36.532000+00:00",
  "ClusterIdentifier": "redshift-cluster-1",
  "DbUser": "awsuser",
  "Database": "dev"
}
```

Listing 10: get-statement-result partial output (Task 6)

```
{
```

```
  "Records": [[
    {"longValue": 3},
    {"stringValue": "IFT66TXU"},
    {"stringValue": "High Point"},
    {"stringValue": "ME"},
    {"booleanValue": true},
    {"booleanValue": false},
    {"isNull": true},
    {"booleanValue": false},
    {"isNull": true}
  ]],
  "TotalNumRows": 1
}
```


B CLI Output — Mary's Access Verification (Task 8)

Listing 11: execute-statement output with Mary's credentials


```
{
  "Id": "a8733a19-e052-40c1-83ef-965857143d70",
  "CreatedAt": "2026-03-07T23:50:57.851000+00:00",
  "ClusterIdentifier": "redshift-cluster-1",
  "DbUser": "awsuser",
  "Database": "dev"
}
```